

Centro Integrado de Formación Profesional nº 1

UT 2 Programación de Hilos

Curso: DAMSV_2A 2026-2027

Profesor: Miguel Ángel García Cejudo (miguelangel.garcia@educeuta.es)

Objetivos

- Identificar situaciones en las que puede ser ventajoso desarrollar una aplicación multihilo.
- Conocer las técnicas básicas de programación para desarrollar aplicaciones multihilo.
- Crear y lanzar, desde el hilo inicial de un programa, varios hilos que compartan información.
- Asegurar la exclusión mutua en el acceso a la información compartida por varios hilos.
- Utilizar técnicas específicas para evitar el interbloqueo.
- Usar, de manera efectiva, técnicas de espera no activa cuando deba suspenderse la ejecución de un hilo a la espera de que se den las condiciones para que pueda continuar su ejecución.
- Aprender los mecanismos que proporciona el lenguaje Java para esperar a que un hilo termine su ejecución, para interrumpir su ejecución y para establecer su prioridad.
- Desarrollar hilos que no hagan más uso del procesador del estrictamente necesario, permitiendo un mayor grado de concurrencia entre los distintos hilos de un programa.
- Emplear, de manera efectiva, un entorno de ejecución para depurar aplicaciones multihilo, y para detectar e investigar interbloqueos.

Programación de Hilos

2.1. Programación de hilos en Java

En este capítulo se aprenderá a crear y lanzar nuevos hilos o hebras (**threads** en inglés) de un proceso con Java, y a hacer que diferentes hilos de un mismo proceso colaboren y se coordinen entre sí para realizar una tarea.

La ejecución de un proceso comienza con un único hilo, pero se pueden crear más sobre la marcha. Los distintos hilos de un mismo proceso comparten:

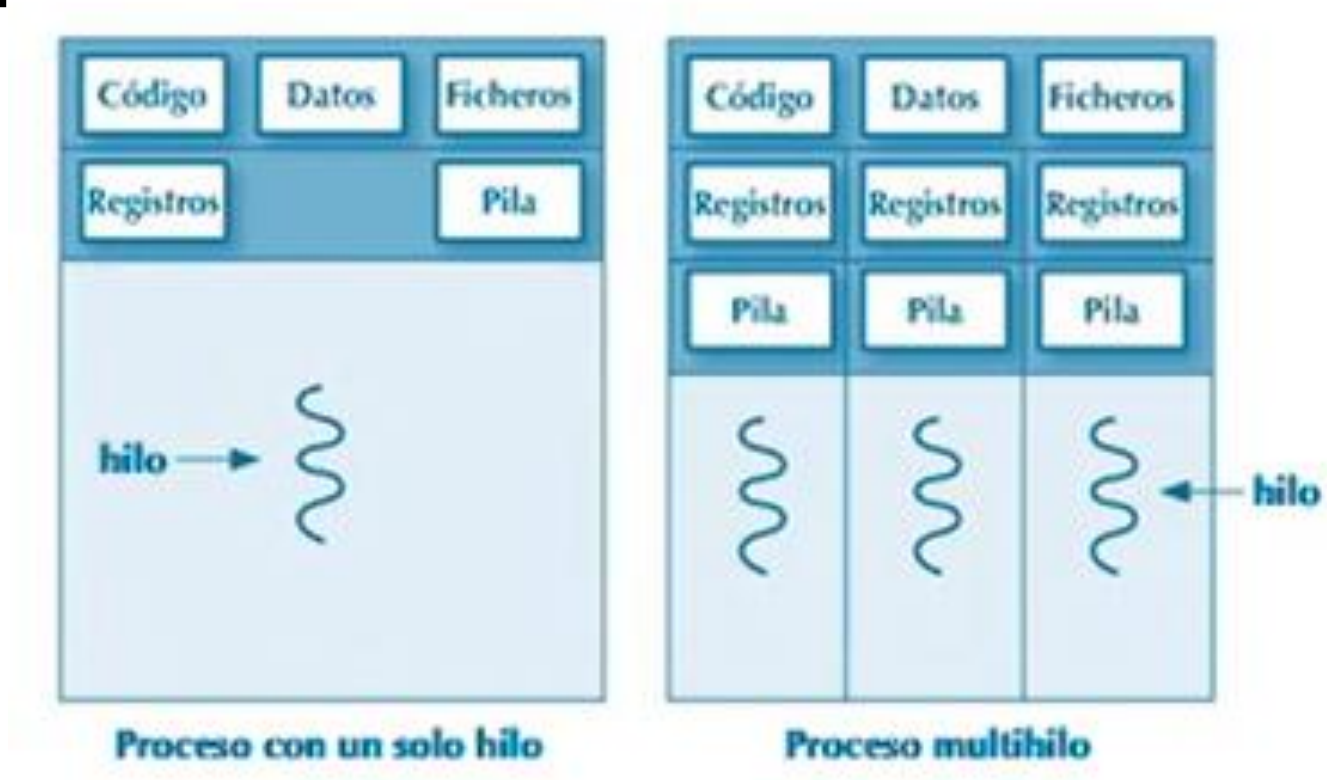
- El espacio de memoria asignado al proceso.
- La información de acceso a ficheros. Estos se utilizan no solo para almacenar datos, sino también para controlar dispositivos de entrada y salida (E/S).

En cambio, cada hilo tiene sus propios valores para:

- Los registros del procesador.
- El estado de su pila (stack). En la pila se guarda información acerca de las llamadas en curso de ejecución a métodos de diversos objetos. Para cada llamada se guardan, entre otras cosas, los datos locales (en variables internas del método).

Programación de Hilos

2.1. Programación d



El intercambio de información entre hilos es sencillo, dado que los distintos hilos de un mismo proceso comparten la memoria asignada al proceso por el sistema operativo. Pero los distintos hilos de un mismo proceso deben coordinarse para el acceso a los contenidos de la memoria y a los ficheros que se utilizan para controlar las operaciones de E/S.

Programación de Hilos

2.1. Programación de hilos en Java

En el resto de este capítulo se verán los mecanismos y las clases que proporciona el lenguaje Java para la programación de hilos y su sincronización.

Se pueden utilizar hilos en situaciones muy diversas, como por ejemplo las siguientes:

- **Programas servidores** que proporcionan servicios a otros procesos, como por ejemplo un servidor web o de bases de datos. Cada petición al servidor se puede atender en un hilo distinto. De esa manera, se pueden atender simultáneamente muchas peticiones de servicio, y se mejora el tiempo de respuesta para ellas. En el capítulo siguiente se aprenderá a programar servidores multihilo que prestan servicios a otros procesos a través de protocolos estándar de red, y que crean sobre la marcha nuevos hilos para atender nuevas peticiones.
- **Programas de todo tipo** que muestran una interfaz gráfica de usuario a la vez que realizan procesos en segundo plano. Es el caso de un procesador de texto o de una hoja de cálculo. En una hoja de cálculo, un hilo podría responder a las acciones del usuario sobre la interfaz de usuario. Otro podría recalcular los valores que cambian en cualquier celda como consecuencia del cambio en el valor de una celda, y volver a mostrar los nuevos valores donde hayan cambiado. Otro podría ocuparse de guardar el documento periódicamente, y otro podría imprimir el documento.

Programación de Hilos

2.1. Programación de hilos en Java

En el resto de este capítulo se verán los mecanismos y las clases que proporciona el lenguaje Java para la programación de hilos y su sincronización.

Se pueden utilizar hilos en situaciones muy diversas, como por ejemplo las siguientes:

- **Juegos** en los que intervienen muchos elementos que interactúan entre ellos, y que pueden ser controlados por los usuarios. En lugar de un único hilo que controla todos los elementos del juego, se pueden tener distintos hilos para distintos elementos del juego.
- **Programas de control en tiempo real**. Estos controlan un sistema mediante dispositivos sensores y actuadores. Obtienen información del estado del sistema a través de dispositivos sensores, y actúan sobre él mediante dispositivos actuadores. Un ejemplo sería un programa de control del acceso a un aparcamiento, que debe detectar la llegada de vehículos mediante determinados sensores, y debe actuar sobre el sistema mostrando información para los conductores en una pantalla, haciendo que se generen los tickets, y levantando y bajando una barrera en momentos determinados. Existen muchos dispositivos que intervienen en este proceso, y se pueden controlar mediante distintos hilos que se comunican y coordinan entre sí.

Programación de Hilos

2.2. Creación de hilos en Java

Para ejecutar un programa de Java, se inicia la máquina virtual de Java (JVM) con un único hilo que ejecuta el método **main**() de su clase. Pero desde este hilo se pueden crear más hilos.

- Los hilos se pueden crear en Java a partir de cualquier clase que implemente la interfaz **Runnable**. En esta clase hay que programar el método **run**(), que se ejecuta cuando se lanza el hilo.
- Un hilo se gestiona con un objeto de la clase **Thread**. Después de crearlo, se puede lanzar con su método **start**(). Con esto se crea el hilo y se ejecuta el método start().
- El siguiente programa de ejemplo lanza dos hilos de un proceso. La funcionalidad de los hilos se implementa en una clase Hilo, que implementa la interfaz Runnable. Al lanzar el hilo se ejecuta el método run(), que escribe un identificador que se pasa en el constructor.

Programación de Hilos

2.2. Creación de hilos en Java


```java
package lanzahilos;

class Hilo implements Runnable {
    private final String nombre;

    Hilo(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void run() {
        System.out.printf("Hola, soy el hilo: %s.\n", this.nombre);
        System.out.printf("Hilo %s terminado.\n", this.nombre);
    }
}

public class LanzaHilos {
    public static void main(String[] args) {
        Thread h1 = new Thread(new Hilo("H1"));
        Thread h2 = new Thread(new Hilo("H2"));
        h1.start();
        h2.start();
        System.out.println("Hilo principal terminado.");
    }
}
```

Programación de Hilos

2.2. Creación de hilos en Java

Si se ejecuta el programa varias veces, se puede ver que, en cada ejecución, las instrucciones de los tres hilos (el principal y los dos creados por él) intercalan su ejecución de diversas maneras, y empiezan y terminan su ejecución en distintos órdenes.

Programación de Hilos

2.3. La clase Thread

La clase **Thread** proporciona la funcionalidad esencial para la creación, ejecución y gestión de hilos.

La tabla resume los principales métodos de esta clase.

Método	Funcionalidad
void run():	Se ejecuta cuando se lanza el hilo. Es el punto de entrada del hilo, como el método main() es el punto de entrada del proceso.
void start():	Lanza el hilo. La JVM crea el hilo y ejecuta su método run().
static void sleep(long ms) / static void sleep(long ms, long ns):	Detiene la ejecución del hilo actualmente en ejecución durante un tiempo, que se puede indicar en microsegundos o en una combinación de microsegundos y nanosegundos
void join() / void join(long ms) / void join(long ms, long ns):	Espera a que termine el hilo. Se puede indicar un tiempo máximo de espera, bien en milisegundos, bien en una combinación de milisegundos y nanosegundos.
public void interrupt(): public boolean isInterrupted(): public static boolean interrupted():	El primer método interrumpe la ejecución de un hilo. El segundo método verifica si se ha interrumpido un hilo. El tercer método (estático) verifica si se ha interrumpido el hilo actual, y borra el estado de interrupción, de manera que una llamada posterior devolvería false, a menos que se vuelva a interrumpir.

Programación de Hilos
2.3. La clase Thread

Método	Funcionalidad
boolean isAlive():	Comprueba si el hilo está vivo. Un hilo está vivo cuando se ha iniciado y no ha terminado su ejecución
int getPriority() / void setPriority(int nuevaPrior):	Se puede asignar una prioridad a un hilo, y se puede obtener la prioridad de un hilo.
static Thread currentThread():	Devuelve un objeto de clase Thread correspondiente al hilo en ejecución actualmente
long getId():	Devuelve el identificador del hilo.
String getName() / void setName(String nombre):	Se puede asignar un nombre a un hilo, y se puede recuperar el nombre del hilo.
Thread.State getState():	Devuelve el estado del hilo.
boolean isDaemon() / void setDaemon(boolean on):	Un hilo puede ser de tipo daemon. La distinción es importante, porque la JVM termina su ejecución cuando no queda ningún hilo activo o cuando solo quedan hilos de tipo daemon.

Programación de Hilos
2.3. La clase Thread

La clase **Thread** tiene multitud de métodos para gestión de hilos. Pero muchos de ellos, como bien se explica en la propia documentación de la clase, están obsoletos (deprecated) o plantean problemas importantes en muchos escenarios de uso, por lo que está totalmente desaconsejado su uso, que, además, es en general innecesario para resolver problemas de programación concurrente. Con los métodos y las técnicas que se explican y utilizan en este capítulo, se pueden resolver la práctica totalidades de los problemas de programación concurrente que se puedan plantear.

Programación de Hilos
2.3. La clase Thread

Actividades propuestas

2.1. Escribe un programa que obtenga el objeto Thread correspondiente al hilo actual de ejecución y muestre información acerca de él. Este programa no tiene que crear ningún hilo, solo mostrar información sobre el hilo actualmente en ejecución.

2.2. Averigua la mínima y máxima prioridad que puede tener un hilo. Consulta para ello la documentación de la clase Thread, y escribe un programa que muestre esta información.

Programación de Hilos
2.3. La clase Thread

El siguiente programa lanza dos hilos. Cada uno de ellos hace pausas de duración aleatoria de entre 10 y 500 ms, utilizando el método sleep de la clase Thread. El hilo principal utiliza el método **join** para esperar a que terminen los dos hilos lanzados, por lo que siempre terminará el último. Los dos métodos anteriores pausarán la ejecución del hilo, y durante ese periodo de tiempo se podría interrumpir. Si esto sucede, se lanzaría una InterruptedException, que se captura para mostrar un mensaje.

Programación de Hilos
2.3. La clase Thread


```
import java.util.Random;

class Hilo_1 implements Runnable {
    private final String nombre;

    Hilo_1(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void run() {
        System.out.printf("Hola, soy el hilo: %s.\n", this.nombre);
        Random r = new Random();
        for (int i = 0; i < 5; i++) {
            int pausa = 10 + r.nextInt(500 - 10);
            System.out.printf("Hilo: %s hace pausa de %d ms.\n", this.nombre, pausa);
            try {
                Thread.sleep(pausa);
            } catch (InterruptedException e) {
                System.out.printf("Hilo %s interrumpido.\n", this.nombre);
            }
        }
        System.out.printf("Hilo %s terminado.\n", this.nombre);
    }
}
```

Programación de Hilos
2.3. La clase Thread


```
public class LanzaHilosYEsperaQueTerminen {
    public static void main(String[] args) {
        Thread.class // (Nota: visualmente corresponde al main lanzando los hilos h1 y
h2)
        Thread h1 = new Thread(new Hilo_("H1"));
        Thread h2 = new Thread(new Hilo_1("H2"));
        h1.start();
        h2.start();
        try {
            h1.join();
            h2.join();
        } catch (InterruptedException e) {
            System.out.println("Hilo principal interrumpido.");
        }
        System.out.println("Hilo principal terminado.");
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos

Un programa puede lanzar múltiples hilos que colaboren entre sí para la realización de una tarea. Deben utilizarse mecanismos de sincronización para evitar problemas que se puedan dar en determinadas situaciones. Como cuando cooperan personas, el problema de fondo es que alguien hace algo cuando no hay que hacerlo. Y la solución pasa por esperar hasta que llegue el momento de hacerlo.
En el programa de ejemplo anterior, todos los hilos realizan tareas completamente independientes. Normalmente no es así, sino que los diferentes hilos colaboran para realizar un trabajo conjunto. El siguiente programa de ejemplo lanza varios hilos que cooperan incrementando un contador hasta un valor determinado. Ese valor se divide entre el número de hilos, y el resultado es la cantidad de veces que cada hilo debe incrementarlo. El contador se implementa como un objeto de una clase Contador, que se pasa a cada hilo en su constructor. De esta forma, todos los hilos comparten este objeto. Todos pueden consultar su valor y todos pueden modificarlo. El hilo principal crea y lanza todos los hilos y espera a que terminen de ejecutarse utilizando el método join

Programación de Hilos
2.4. Sincronización de hilos


```
class Hilo_2 implements Runnable {
    int numHilo, miParte, miCuenta = 0;
    private final Contador cont;

    public int getMiCuenta() {
        return miCuenta;
    }

    Hilo_2(int numHilo, int miParte, Contador c) {
        this.numHilo = numHilo;
        this.miParte = miParte;
        this.cont = c;
    }

    @Override
    public void run() {
        for (int i = 0; i < miParte; i++) {
            this.cont.incrementa();
            miCuenta++;
        }
        System.out.printf("Hilo %d terminado, cuenta: %d\n", numHilo, getMiCuenta());
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos


```
class Contador {
    private int cuenta = 0;

    public int getCuenta() {
        return cuenta;
    }

    public int incrementa() {
        this.cuenta++;
        return cuenta;
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos


```
class HilosCooperantesNoSincr {
    private static final int NUM_HILOS = 10;
    private static final int CUENTA_TOTAL = 100000;

    public static void main(String[] args) {
        Contador c = new Contador();
        Thread[] hilos = new Thread[NUM_HILOS];
        for (int i = 0; i < NUM_HILOS; i++) {
            Thread th = new Thread(new Hilo_2(i, CUENTA_TOTAL / NUM_HILOS, c));
            th.start();
            hilos[i] = th;
        }
        for (Thread h : hilos) {
            try {
                h.join();
            } catch (InterruptedException e) {
            }
        }
        System.out.printf("Cuenta global: %s\n", c.getCuenta());
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos

Con el programa anterior, el contador no llega al valor que cabría esperar, dado por la variable estática de clase CUENTA_TOTAL. Se puede probar con distintos valores para ella. Es posible que llegue para valores pequeños, pero la discrepancia es mayor cuanto mayor sea su valor.
En el siguiente apartado se explica el motivo y se da una sencilla solución.

Actividad propuesta 2,3
¿Qué pasaría si el programa anterior no se llamara al método join? ¿Y si se hiciera en el mismo bucle que lanza los hilos con start?

2.4. Sincronización de hilos

2.4.1. Exclusión mutua. Condiciones de carrera y secciones críticas

El programa anterior no funciona, porque la operación que realiza el método incrementa no es atómica, sino que se descompone en operaciones más elementales que se ejecutan una tras otra

```
public int incrementa() {
    this.cuenta++;
    return cuenta;
}
```

Todas las operaciones aritméticas se realizan en registros del procesador. Para realizarlas sobre datos que están en la memoria, antes hay que pasarlos a un registro del procesador y, una vez realizadas, hay que pasar el resultado a la memoria. El código resultante de compilar la sentencia this.cuenta++ del método incrementa es más o menos como sigue:

Programación de Hilos
2.4. Sincronización de hilos

registro <- [cuenta]	Copia valor de la variable cuenta de memoria principal a registro.
incrementar registro	Incrementar el valor de registro.
registro -> [cuenta]	Copia valor de registro a variable cuenta en memoria principal.

Cuando estas operaciones se ejecutan en un hilo, la ejecución del hilo se puede interrumpir y se pueden intercalar entre ellas operaciones de otro hilo.
Java es un lenguaje virtual que se compila a bytecode, que se ejecuta en una máquina virtual. Pero en lo que respecta al problema aquí planteado, esto no supone ninguna diferencia con respecto a un lenguaje compilado, que se compila a código binario, que se ejecuta sobre en máquina física.

Programación de Hilos
2.4. Sincronización de hilos

Según cómo se intercalen las operaciones de los distintos hilos, los resultados pueden no ser los deseados. Esto es lo que se conoce como una **condición de carrera** (race condition en inglés). Existen condiciones de carrera cuando son posibles ejecuciones concurrentes de dos o más procesos que dan lugar a resultados incorrectos. A continuación, se muestra un ejemplo, con la ejecución de las tres instrucciones anteriores para dos hilos H1 y H2 en un mismo procesador. Hay que tener presente que para cada hilo se guarda el estado del procesador, incluyendo el valor de todos los registros, por lo que cada hilo tiene su propio valor para registro, que se restaura en el procesador cuando se pasa a ejecutar el hilo.

Programación de Hilos
2.4. Sincronización de hilos

Resultados incorrectos con ejecución concurrente de dos hilos

Situación inicial: valor en la variable cuenta es 86	
H1 | registro <- [cuenta]	Se guarda valor 86 en registro para H1
H2 | registro <- [cuenta]	Se guarda valor 86 en registro para H2 H1
incrementar registro	El valor de registro pasa a ser 87 para H1 H2
incrementar registro	El valor de registro pasa a ser 87 para H2 H1
registro -> [cuenta]	El valor en la variable cuenta pasa a ser 87
H2 | registro -> [cuenta]	El valor en la variable cuenta pasa a ser 87. Se ha perdido el incremento hecho por H1

Este error no se hubiera producido si las instrucciones de ambos hilos se hubieran intercalado de manera distinta. Los errores debidos a condiciones de carrera pueden ser difíciles de detectar e investigar, porque dependen del orden particular en que se ejecuten las instrucciones de distintos hilos sobre un mismo procesador. Por tanto, no son fácilmente reproducibles. Las tres operaciones anteriores para incrementar el valor del contador se deberían ejecutar sin que entre ellas se intercalen las mismas operaciones de otro hilo.

2.4. Sincronización de hilos

Se dice que constituyen una **sección crítica**. Para evitar condiciones de carrera, las secciones críticas de diferentes hilos deberían ejecutarse con **exclusión mutua**. Es decir, la sección crítica no debería estar ejecutándose concurrentemente en más de un hilo.
Una manera de conseguir la exclusión mutua con Java es con la palabra clave **synchronized**. Basta añadirla a la definición del método incrementa para que este método no se pueda ejecutar concurrentemente para un mismo objeto en más de un hilo. Una clase puede tener más de un método synchronized. No se ejecutará nunca un método synchronized para un objeto en un hilo si ya se está ejecutando cualquier método synchronized en cualquier otro hilo. Se puede verificar que, simplemente añadiendo synchronized al principio de la declaración del método incrementa, la cuenta siempre llega al valor final esperado. El método getCuenta también se declara como synchronized para que no se consulte el valor del contador mientras en otro hilo se esté modificando. Con estos cambios, la clase Contador queda como se muestra a continuación, y se garantiza la exclusión mutua.

Programación de Hilos
2.4. Sincronización de hilos



Centro Integrado de Formación Profesional nº 1

```
class Contador_1 {
    private int cuenta = 0;

    synchronized public int getCuenta() {
        return cuenta;
    }

    synchronized public int incrementa() {
        this.cuenta++;
        return cuenta;
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos

El siguiente programa de ejemplo consta de varios hilos que contabilizan las respuestas para una pregunta. Cada hilo contabiliza las respuestas en una zona, cuyo identificador se le pasa en el constructor. Los encuestados pueden dar cualquier respuesta, o no responder, lo que se contabilizaría como "no sabe/no contesta" (NS/NC). Se quiere contabilizar el número de encuestados para cada zona, y el número de veces que se ha dado cada respuesta en particular, independientemente de la zona. También el número de veces que no se da una respuesta (NS/NC). Los resultados se recopilan en un objeto de clase ResultadosEncuesta, que tiene métodos synchronized para registrar/contabilizar cada respuesta, y para consultar los resultados de la encuesta. Junto con el identificador de su zona, a cada hilo se le pasa el objeto de la clase ResultadosEncuesta, que comparten todos.

2.4. Sincronización de hilos

Para contabilizar el número de veces que se da cada respuesta, se utiliza un HashMap<String, Integer>, que guarda un conjunto de pares (clave, valor), donde la clave es un String que representa una posible respuesta, y el valor es un Integer que contiene el número de veces que se ha dado esa respuesta. La clase HashMap tiene dos características relevantes para esta aplicación.

- La primera es que no es thread-safe, o no para todas las operaciones (los detalles no son importantes para la realización de este programa, pero el lector interesado puede consultar los detalles en la documentación de la clase).
- La segunda es que permite utilizar null como valor para la clave. Asociadas a este valor, se registran las respuestas NS/NC.

El programa principal lanza hilos de recuento para varias zonas. Cada uno proporciona un número de respuestas al azar entre cien y doscientas, y cada respuesta es una respuesta al azar de "respuesta1" a "respuesta9", o bien null, que representa NS/NC

Programación de Hilos
2.4. Sincronización de hilos


```
import java.util.HashMap;
import java.util.Random;
import java.util.Set;

class ResultadosEncuesta {
    private final HashMap<String, Integer> totalPorRespuesta = new HashMap<>();
    private final HashMap<String, Integer> totalPorZona = new HashMap<>();

    // Suma uno a número de respuestas para la zona y para la respuesta
    synchronized public void anotaRespuesta(String idZona, String respuesta) {
        Integer numRespValor = this.totalPorRespuesta.get(respuesta);
        this.totalPorRespuesta.put(respuesta,
                numRespValor == null ? 1 : numRespValor + 1);
        Integer numRespZona = this.totalPorZona.get(idZona);
        this.totalPorZona.put(idZona, numRespZona == null ? 1 : numRespZona + 1);
    }

    synchronized public Set<String> obtenZonas() {
        return this.totalPorZona.keySet();
    }

    synchronized public Set<String> obtenRespuesta() { // (Nota: en la imagen aparece obtenRespuestas())
        return this.totalPorRespuesta.keySet();
    }

    synchronized public int obtenNumRespuestasZona(String zona) {
        return this.totalPorZona.get(zona);
    }

    synchronized public int obtenNumRespuestas(String respuesta) {
        return this.totalPorRespuesta.get(respuesta);
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos


```
class EncuestadorZona implements Runnable {
    private final String idZona;
    private final ResultadosEncuesta resultados;

    EncuestadorZona(String idZona, ResultadosEncuesta resultados) {
        this.idZona = idZona;
        this.resultados = resultados;
    }

    @Override
    public void run() {
        System.out.printf(">>Encuestador para zona %s comienza.\n", this.idZona);
        Random r = new Random();
        int numRespuestas = 100 + r.nextInt(200 - 100) + 1;
        for (int i = 0; i < numRespuestas; i++) {
            int numRespuesta = r.nextInt(10); // Respuesta 0 a 9, 0 NS/NC
            String respuesta = null;
            if (numRespuesta > 0) {
                respuesta = "respuesta_" + numRespuesta;
            }
            this.resultados.anotaRespuesta(this.idZona, respuesta);
        }
        System.out.printf("##Encuestador zona %s termina.\n", this.idZona);
    }
}
```

Programación de Hilos
2.4. Sincronización de hilos


```
public class Encuesta {
    private static final int NUM_ZONAS = 20;

}public static void main(String[] args) {
    ResultadosEncuesta resultados = new ResultadosEncuesta();

    Thread[] encuestadores = new Thread[NUM_ZONAS];
    for (int i = 0; i < NUM_ZONAS; i++) { // Crea hilos
        Thread encuestador = new Thread(
            new EncuestadorZona("zona" + (i + 1), resultados));
        encuestadores[i] = encuestador;
    }
    for (Thread encuestador : encuestadores) { // Lanza hilos
        encuestador.start();
    }
    for (Thread encuestador : encuestadores) { // Espera que terminen
        try {
            encuestador.join();
        } catch (InterruptedException ex) {
        }
    }
    System.out.println("Encuestados por zonas");
    Set<String> zonas = resultados.obtenZonas();
    int granTotalPorZonas = 0;
    for (String zona : zonas) {
        int totalParaZona = resultados.obtenNumRespuestasZona(zona);
        System.out.printf("%s: %d\n", zona, totalParaZona);
        granTotalPorZonas += totalParaZona;
    }
    System.out.printf("TOTAL: %d\n", granTotalPorZonas);
    System.out.println("Resultados por respuesta:");
    Set<String> respuestas = resultados.obtenRespuesta();
    int granTotalPorRespuestas = 0;
    for (String respuesta : respuestas) {
        int totalParaRespuesta = resultados.obtenNumRespuestas(respuesta);
        System.out.printf("%s: %d\n",
            respuesta != null ? respuesta : "NS/NC", totalParaRespuesta);
        granTotalPorRespuestas += totalParaRespuesta;
    }
```

Programación de Hilos

2.4. Sincronización de hilos

Actividad propuesta 2.4

Un hilo debe generar un número al azar entre cero y cien, que deben intentar adivinar otros diez hilos. Si un hilo acierta el número, debe terminar su ejecución inmediatamente. Y el resto de los hilos deben también terminar su ejecución en cuanto propongan un número y se les avise de que otro hilo ya ha acertado el número.

Se propone utilizar una clase NumeroOculto con un método int propuseNumero(int num) que devuelva los siguientes valores:

a) -1 si el juego ya ha terminado porque un hilo ha adivinado el número.
b) 1 si el número propuesto (num) es el número oculto.
c) 0 en otro caso.

No hace falta crear una clase para el hilo que genera el número al azar. Es el hilo inicial, que ejecuta el método main, y que crea el resto de los hilos.

Programación de Hilos

2.4. Sincronización de hilos

2.4.2. Bloqueo intrínseco, bloques de código sincronizados

La implementación de la palabra clave synchronized se basa en el mecanismo del bloqueo intrínseco (intrinsic lock). Cada objeto de Java posee un bloqueo intrínseco por el solo hecho de serlo, es decir, por pertenecer a la clase Object.

Cada bloqueo intrínseco tiene asociada una cola de procesos a la espera de adquirirlo. Por otra parte, nada impide a un hilo adquirir de nuevo un bloqueo intrínseco que ya ha adquirido. Esto se llama sincronización reentrante.

La palabra clave synchronized puede aplicarse en distintos tipos de bloques de código de Java, y en cada caso se utilizará un objeto de bloqueo distinto.

Programación de Hilos

2.4. Sincronización de hilos

2.4.2. Bloqueo intrínseco, bloques de código sincronizados

- **Métodos no estáticos**. Como en los ejemplos anteriores, se añade synchronized a la declaración del método. El bloqueo intrínseco se realiza sobre el objeto sobre el que se ejecuta el método (this). Por tanto, dos hilos distintos no pueden estar a la vez ejecutando cada uno un método no estático sincronizado sobre el mismo objeto.

- **Métodos estáticos**. El bloqueo intrínseco se realiza sobre la clase a la que pertenece el método. En Java, una clase es también un objeto. De hecho, es una instancia de la clase Class. Por tanto, dos hilos distintos no pueden estar a la vez ejecutando cada uno un método estático sincronizado de la misma clase.

- **Bloques de código cualquiera**. Después de la palabra clave synchronized, y entre paréntesis, se indica el objeto sobre el que se realiza el bloqueo:

```
synchronized(objeto) {
    (...)
}
```

2.4. Sincronización de hilos

2.4.2. Bloqueo intrínseco, bloques de código sincronizados

Un hilo no puede ejecutar un bloque de código sincronizado sobre un objeto si ya existe un hilo que ha adquirido el bloqueo para dicho objeto. El objeto debería ser de tipo final, porque si se le asigna un nuevo valor, quedan sin efecto todos los bloqueos que existen sobre dicho objeto.

Un objeto de bloqueo debería ser de tipo final, porque si se le asigna un nuevo valor, quedan sin efecto todos los bloqueos que existen sobre dicho objeto. A un objeto del tipo final, una vez que se ha creado y se le ha asignado un valor, no se le puede asignar un nuevo valor.

Programación de Hilos

2.4. Sincronización de hilos

2.4.2. Bloqueo intrínseco, bloques de código sincronizados

Si la ejecución de un hilo llega a un elemento del lenguaje en el que está la palabra clave synchronized, y el bloque o recurso intrínseco para el objeto correspondiente está libre, se otorga el bloqueo al hilo. Una vez ejecutado el código del elemento del lenguaje, se libera el bloqueo. Si el hilo ya tiene el bloqueo, podrá volver a ejecutar sin más el código del elemento del lenguaje (bloqueo reentrante).

En el ejemplo anterior se han visto métodos no estáticos sincronizados. Ahora se muestra un ejemplo con bloques de código sincronizados. Estos permiten un control más fino de la sincronización. En el siguiente ejemplo se tiene una clase que controla dos contadores. Hay que evitar que dos hilos diferentes realicen modificaciones a la vez sobre el mismo contador, pero no hay ningún problema en que dos hilos diferentes modifiquen cada uno un contador distinto. Sin embargo, esto último no se permitiría si sencillamente se declarasen los métodos que incrementan ambos contadores como synchronized. En este ejemplo se utiliza un objeto de sincronización distinto para cada contador.

Programación de Hilos

2.4. Sincronización de hilos

2.4.2. Bloqueo intrínseco, bloques de código sincronizados



Centro Integrado de Formación Profesional nº 1

```
class Contadores {

    private long cont1 = 0;
    private long cont2 = 0;

    private final Object lock1 = new Object();
    private final Object lock2 = new Object();

    public void incrementar1() {
        synchronized (lock1) {
            cont1++;
        }
    }

    public long getContador1() {
        synchronized (lock1) {
            return cont1;
        }
    }

    public void incrementar2() {
        synchronized (lock2) {
            cont2++;
        }
    }

    public long getContador2() {
        synchronized (lock2) {
            return cont2;
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

En este apartado se verá otro problema típico de la programación concurrente: el de la compartición de recursos. Existen distintos hilos que utilizan recursos compartidos. Para realizar determinadas operaciones, los hilos necesitan obtener, previamente, acceso en exclusiva a un conjunto de recursos. Una vez terminada la operación, liberan los recursos, que pueden entonces ser utilizados por otros hilos.

Los recursos se representan mediante objetos, y se obtiene acceso en exclusiva a ellos mediante bloques synchronized. Si se utiliza más de uno, se utilizan bloques synchronized anidados. Si fueran dos, se haría de la siguiente forma:

```
synchronized (r1) {
    synchronized (r2) {
        // realizar operaciones con r1 y r2
    }
}
```

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Los recursos se obtienen en el orden en que aparecen los bloques synchronized anidados. En el ejemplo anterior, cuando se obtiene el bloqueo sobre el primer recurso, r1, se queda a la espera de obtener el bloqueo sobre el segundo, r2.

Puede producirse un interbloqueo (deadlock) cuando dos hilos quedan mutuamente bloqueados, cada uno a la espera de que el otro desbloquee un objeto que ha bloqueado.

Las asignaciones de recursos pueden representarse en forma de grafos de asignación de recursos. En ellos se representan los hilos con cuadrados y los recursos, con círculos. La solicitud de un recurso, por parte de un hilo, se representa con una flecha del hilo al recurso. La asignación de un recurso a un hilo se representa con una flecha del recurso al hilo.

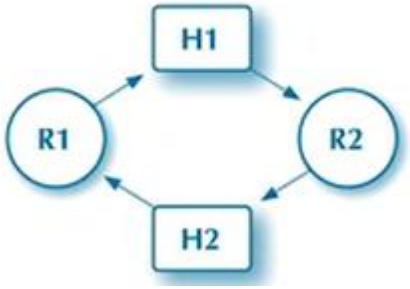
Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Existe interbloqueo cuando se crea un ciclo en el grafo de asignación de recursos. En este ejemplo, el proceso H1 tiene asignado el recurso R1, está a la espera de conseguir el recurso R2 y no liberará R1 mientras no consiga R2. Pero R2 está asignado a H2, que está a la espera de conseguir R1, y no liberará R2 mientras no consiga R1. Existe un bucle en el grafo de asignación de recursos y, por tanto, hay un interbloqueo entre todos los hilos involucrados en el ciclo, en este caso, H1 y H2.

Una forma sencilla de evitar el interbloqueo es establecer un orden entre los recursos, y que los hilos soliciten los recursos según ese orden. En este caso, podría ser primero R1 y luego R2. Si tanto H1 como H2 solicitan primero R1 y después R2, no se puede dar el interbloqueo.



Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Un ejemplo de compartición de recursos entre hilos es un sistema para transferencias bancarias. Para realizar una transferencia de una cuenta c1 a una cuenta c2, debe conseguirse el acceso en exclusiva a ambas cuentas, ya que, mientras se está realizando la transferencia, ningún otro proceso debe acceder a ninguna de las dos cuentas, ni siquiera para consultar su saldo. El método que realiza estas operaciones podría ser como sigue.

```
public boolean transferencia(Cuenta c1, Cuenta c2, int cantidad) {
    boolean result = false;
    synchronized(c1) {
        synchronized(c2) {

            if (c1.getSaldo() >= cantidad) {
                c1.sacar(cantidad);
                c2.ingresar(cantidad);
                result = true;
            }
        }
    }
    return result;
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

En este caso, el interbloqueo se puede producir cuando un hilo H1 intenta una transferencia de una cuenta C1 a una cuenta C2, y otro hilo H2 intenta una transferencia de C2 a C1. Si H1 adquiere el bloqueo intrínseco de C1 y, acto seguido, P2 adquiere el de C2, entonces se producirá interbloqueo entre ambos hilos: H1 estará bloqueado a la espera de conseguir el bloqueo de C2, y H2 estará bloqueado a la espera de conseguir el bloqueo de C1.

Ya se ha comentado que este problema se puede solucionar estableciendo un orden entre los recursos, y reservando los recursos siempre en ese orden. El orden entre las cuentas podría ser, por ejemplo, el orden alfabético del número de cuenta.

Se puede utilizar una clase auxiliar GestorTransferencias para realizar la transferencia entre dos cuentas y simplificar así el código en programas que tengan que realizar transferencias. Esta clase realiza siempre las transferencias de la que es menor, por orden alfabético, a la que es mayor. Si no se puede realizar una transferencia porque el saldo en la cuenta de origen es insuficiente, sencillamente se desiste y se libera el bloqueo sobre las dos cuentas.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo



Centro Integrado de Formación Profesional nº 1

```
public class GestorTransferencia {
    public static boolean transferencia(Cuenta_2 c1,
            Cuenta_2 c2,
            int cantidad) {
        Cuenta_2 cuentaMenor, cuentaMayor;
        if (c1.getNumCuenta()
                .compareTo(c2.getNumCuenta()) < 0) {
            cuentaMenor = c1;
            cuentaMayor = c2;
        } else {
            cuentaMenor = c2;
            cuentaMayor = c1;
        }
        boolean result = false;
        synchronized (cuentaMenor) {
            synchronized (cuentaMayor) {
                if (c1.getSaldo() >= cantidad) {
                    c1.sacar(cantidad);
                    c2.ingresar(cantidad);
                    result = true;
                }
            }
        }
        return result;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Se muestra a continuación la clase Cuenta. Los métodos de esta clase que modifican o consultan el estado de la cuenta son métodos synchronized. Pero no es necesaria la sincronización para numCuenta, dado que nunca se modifica su valor una vez asignado.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo



Centro Integrado de Formación Profesional nº 1

```
public class Cuenta_2 {
    int saldo;
    final String numCuenta;
    public Cuenta_2(String numCuenta, int saldoInicial) {
        this.saldo = saldoInicial;
        this.numCuenta = numCuenta;
    }
    public synchronized int getSaldo() {
        return this.saldo;
    }
    public synchronized void ingresar(int cantidad) {
        this.saldo += cantidad;
    }
    public synchronized void sacar(int cantidad) {
        this.saldo -= cantidad;
    }
    public String getNumCuenta() {
        return this.numCuenta;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

En la clase principal está el método main, que crea los hilos, los lanza y espera a que terminen.
Un hilo realiza transferencias de la cuenta C1 a la cuenta C2, y otro de C2 a C1.

```
class Hilo_3 implements Runnable {

    Cuenta_2 c1, c2;
    String nomHilo;

    Hilo_3(Cuenta_2 c1, Cuenta_2 c2, String nomHilo) {
        this.c1 = c1;
        this.c2 = c2;
        this.nomHilo = nomHilo;
    }
    @Override
    public void run() {
        int cantidad = 10;
        int numTransf = 0;
        for (int i = 0; i < 10000; i++) {

            if (GestorTransferencia.transferencia(c1, c2, cantidad)) {
                numTransf++;
            }
        }
        System.out.printf(
            "**** FIN de %s, %d transferencias hechas de %s a %s.\n",
                this.getNomHilo(),
                numTransf,
                c1.getNumCuenta(),
                c2.getNumCuenta());
    }
    public String getNomHilo() {
        return nomHilo;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

En la clase principal está el método main, que crea los hilos, los lanza y espera a que terminen.
Un hilo realiza transferencias de la cuenta C1 a la cuenta C2, y otro de C2 a C1.

```
class Hilo_3 implements Runnable {

    Cuenta_2 c1, c2;
    String nomHilo;

    Hilo_3(Cuenta_2 c1, Cuenta_2 c2, String nomHilo) {
        this.c1 = c1;
        this.c2 = c2;
        this.nomHilo = nomHilo;
    }
    @Override
    public void run() {
        int cantidad = 10;
        int numTransf = 0;
        for (int i = 0; i < 10000; i++) {

            if (GestorTransferencia.transferencia(c1, c2, cantidad)) {
                numTransf++;
            }
        }
        System.out.printf(
            "**** FIN de %s, %d transferencias hechas de %s a %s.\n",
                this.getNomHilo(),
                numTransf,
                c1.getNumCuenta(),
                c2.getNumCuenta());
    }
    public String getNomHilo() {
        return nomHilo;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo



Centro Integrado de Formación Profesional nº 1

```
public class TransferenciasSinBloqueo {

    public static void main(String[] args) {

        Cuenta_2 c1 = new Cuenta_2("ES1512345678901234567890", 12500);
        Cuenta_2 c2 = new Cuenta_2("ES4578901234567890123456", 23400);

        System.out.printf("Saldo inicial de %s: %d\n",
                c1.getNumCuenta(),
                c1.getSaldo());

        System.out.printf("Saldo inicial de %s: %d\n",
                c2.getNumCuenta(),
                c2.getSaldo());

        System.out.println("--------------------------------");

        // Dos hilos: uno hace transferencias de c1 a c2,
        // otro de c2 a c1

        Thread h1 = new Thread(new Hilo_3(c1, c2, "H1"));
        Thread h2 = new Thread(new Hilo_3(c2, c1, "H2"));

        h1.start();
        h2.start(); TransferenciasSinBloqueo is public, should be
declared in a file named TransferenciasSinBloqueo.
        try {
            h1.join();
            h2.join();
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }

        System.out.println("--------------------------------");

        System.out.printf("Saldo final %s: %d\n",
                c1.getNumCuenta(),
                c1.getSaldo());

        System.out.printf("Saldo final %s: %d\n",
                c2.getNumCuenta(),
                c2.getSaldo());
    }
}
```

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Normalmente, las transferencias no se realizarían de manera tan repetida y, por lo tanto, sería más difícil que se produjera interbloqueo, en el caso de que el bloqueo de las cuentas no fuera siempre en el mismo orden. Si se modifica la clase GestorTransferencias, para que realice el bloqueo de las cuentas en el orden en que se le dan, se puede comprobar que es muy fácil que se produzca el interbloqueo. Pero si se introduce una espera de duración aleatoria entre sucesivas transferencias realizadas por cada hilo, es mucho más difícil que se produzca el interbloqueo, aunque todavía podría suceder alguna vez. Este tipo de errores, que suceden solo ocasionalmente, no son sencillos de detectar con una prueba rutinaria. La cuestión es que no son fácilmente reproducibles. Los programas deben diseñarse y codificarse cuidadosamente para evitar interbloqueos.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Actividad propuesta 2.7
Prueba el programa anterior para transferencia entre cuentas. Ejecútalo varias veces para verificar que no se produce interbloqueo. De todas formas, como ya se ha explicado, unas pocas ejecuciones no son suficiente garantía, porque los interbloqueos, si se dan, podrían suceder de manera esporádica.
Modifica la clase GestorTransferencias para que el bloqueo de las cuentas se haga en el mismo orden en que se dan las cuentas, y no siempre primero la menor y después la mayor. Verifica que se produce interbloqueo y no termina la ejecución del programa.

Introduce, para cada hilo, una espera aleatoria de pocos milisegundos entre transferencia y transferencia. Observa cómo es mucho más difícil que se produzca el interbloqueo.

Puedes utilizar las funcionalidades que proporciona el entorno de desarrollo para depurar (debug) programas multihilo y detectar interbloqueos. Al final del capítulo se dedica una sección a ello.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Actividad propuesta 2.8

Hay cuatro estudiantes que comparten nueve libros. Los libros se pueden almacenar en un array:
Libro[] libros = new Libro[9];
Los estudiantes seleccionan dos libros al azar y, una vez obtenidos (para lo que podrían tener que esperar si, en ese momento, hay otro estudiante utilizando alguno de ellos), los utilizan durante un tiempo aleatorio de entre una y tres horas. Pasado este tiempo, los devuelven, de manera que puedan utilizarlos otros estudiantes, y descansan un tiempo aleatorio de entre una y dos horas. Entonces, vuelven a seleccionar otros dos libros y continúa el ciclo. La clase Libro puede no tener ninguna funcionalidad especial, más allá de tener un identificador, que puede utilizarse en los mensajes en los que se haga referencia al libro. La simulación no tiene por qué ser en tiempo real. Se pueden seleccionar los tiempos al azar en minutos en lugar de horas, y esperar un milisegundo por cada minuto.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

Actividad propuesta 2.9

Existen varias actividades de reparación que necesitan determinadas herramientas para llevarse a cabo, a saber: un taladro, un destornillador y unos alicates. De estas herramientas solo hay una de cada tipo.
La actividad A1 necesita un destornillador y un taladro.
La actividad A2 necesita un destornillador y unos alicates.
La actividad A3 necesita un taladro, un destornillador y unos alicates.
Las actividades las realizan, continuamente, un conjunto de agentes.

Existen dos agentes que realizan la actividad A1, un agente que realiza la actividad A2 y un agente que realiza la actividad A3.

Cada actividad tarda en realizarse un tiempo de entre 50 y 200 ms.
Realiza una simulación de estas actividades. Las actividades se realizan de forma indefinida.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

El planteamiento antes propuesto para evitar el interbloqueo tiene dos inconvenientes:

1. No siempre existe un identificador que permita establecer una ordenación entre las distintas instancias de un mismo objeto.

2. Los recursos pueden ser de distinto tipo, al contrario de lo que sucede en el ejemplo anterior, en que los únicos recursos compartidos son cuentas bancarias.

El primero admite una solución general con el lenguaje de programación Java. Consiste en asignar a cada objeto un identificador único en el momento de su creación, cuyo valor se obtiene de una variable estática de clase. A esta variable se le asigna un valor inicial, por ejemplo, 0. En los creadores de la clase se asigna el valor actual al objeto creado, y después se incrementa su valor. Por supuesto, es necesario que el acceso a esta variable esté protegido frente a accesos concurrentes por parte de diferentes hilos. Esto se puede hacer con lo aprendido hasta ahora. Pero es mucho más sencillo utilizar las clases AtomicInteger y AtomicLong, que son thread-safe, es decir, que se pueden utilizar directamente y de forma segura en programas multihilo, ya que la propia clase implementa los mecanismos de sincronización necesarios para su acceso concurrente por parte de distintos hilos.

Programación de Hilos

2.4. Sincronización de hilos

2.4.3. Compartición de recursos. Interbloqueo

El segundo problema se puede solucionar estableciendo una ordenación entre los tipos de recursos, que con una aplicación en Java, serían las clases a las que pertenecen los objetos compartidos. Los objetos de unas clases se reservarían (bloquearían) antes que los de otras. Y dentro de una clase, se bloquearían según el orden establecido para la clase.

Actividad propuesta 2.10

Implementa el mecanismo sugerido anteriormente para la clase Cuenta. Debe asignarse a cada nueva instancia un valor distinto para un miembro de datos de tipo long. Su valor se debe asignar utilizando un miembro estático de tipo AtomicLong, que se incrementa en el constructor de la clase. Utiliza este identificador, en lugar del número de cuenta, para establecer el orden en que se bloquean las cuentas implicadas en una transferencia. Prueba la aplicación para verificar que no se producen interbloqueos.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

En el apartado anterior se ha visto un tipo de problemas en que varios hilos comparten recursos. Estos recursos podrían ser, sencillamente, objetos cuyo estado puede ser actualizado por cualquiera de estos hilos. Y una vez conseguido el acceso en exclusiva a estos recursos, mediante objetos de bloqueo, se pueden realizar determinadas operaciones, independientemente de ninguna información de estado.

En este apartado se estudia otro tipo de problemas en los que, dependiendo del estado de los recursos, cada uno de los hilos podrá realizar determinadas acciones o no. Y en este último caso, podrá quedar a la espera de que se produzca un cambio de estado, de lo que será notificado por otros hilos.

Para ello, además de un mecanismo de bloqueo sobre los recursos (objetos) compartidos, será necesario un mecanismo de espera para que, en el caso de que el estado de los recursos compartidos no permita que se realice una operación, la ejecución del hilo quede en suspensión la espera de que esa condición se cumpla.

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Y debe ser un mecanismo de espera no activa. Es decir, no se debe consumir tiempo del procesador ni recursos del sistema para comprobar si es posible continuar con la ejecución, mientras no se reciba una notificación de que el estado ha cambiado y podría permitir que el hilo continúe su ejecución.

Para este tipo de problemas, se utilizan varios métodos que se pueden ejecutar solo en un bloque de código synchronized, y sobre el objeto de bloqueo para el bloque de código.

wait()

wait() interrumpe la ejecución del hilo actual. La ejecución del hilo queda bloqueada mientras otro hilo no ejecute el método notify() o el método notifyAll() sobre el objeto. Este método, por tanto, proporciona un mecanismo de espera no activa.

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

notify()

notify() desbloquea uno de los hilos que están esperando sobre el objeto tras haber ejecutado el método wait(), de manera que pueda continuar su ejecución. Este método, por tanto, proporciona un mecanismo de notificación para terminar con la espera no activa de los hilos que están a la espera en un objeto de bloqueo. El orden en que se desbloquean los hilos en un objeto de bloqueo no tiene por qué ser el mismo en que se bloquearon.

notifyAll()

notifyAll() desbloquea todos los hilos que están esperando sobre el objeto de bloqueo tras haber ejecutado el método wait(), de manera que puedan continuar su ejecución.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Estos métodos se utilizan cuando un hilo puede realizar o no una operación dependiendo del estado del sistema. La verificación de ese estado debe hacerse dentro de un método synchronized. Si, de acuerdo con el estado del sistema, no es posible realizar la operación, se ejecuta el método wait() sobre el objeto de bloqueo. Un hilo que cambie el estado del sistema, de manera que podrían continuar otros hilos que han quedado en suspenso tras realizar la operación wait(), ejecuta uno de los métodos notify() o notifyAll() sobre el objeto de bloqueo. Esto permite continuar a uno de los hilos bloqueados en el objeto de bloqueo (en el caso de notify()) o a todos (en el caso de notifyAll()).

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Esquema general

```
synchronized (objetoBloqueo) { // Acceso a recurso con exclusión mutua

        while (!condicionParaOperacion) { // Depende del estado de objetoBloqueo

        try {
        objetoBloqueo.wait(); // Espera que alguien cambie estado y notifique
        } catch (InterruptedException ex) {
        }
    }

    // Se puede realizar operación.
    // No puede interferir otro hilo desde comprobación de condición
    // en while(), porque todo esto está dentro de un bucle synchronized,
    // en el que solo se cede el control con wait()
    realizar_operacion;
    // Si como consecuencia de realizar la operación pudiera continuar
    // otro hilo, habría que hacer objetoBloqueo.notifyAll()
    // u objetoBloqueo.notify()
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Por otra parte, es habitual que objetoBloqueo sea un objeto que contiene el estado compartido por todos los hilos, y sobre el que se puede decidir si distintas operaciones se pueden o no realizar. Por lo tanto, y más específicamente, un esquema típico para los distintos hilos de un programa sería el siguiente:

```
synchronized (objetoEstado) {

    while (!condicionParaOperacion) { // Se comprueba con métodos de objetoEstado

        try {
            objetoEstado.wait(); // Espera que alguien cambie estado y avise
        } catch (InterruptedException ex) {
        }
    }

    // Se puede realizar operación.
    realizar_operacion; // Normalmente modifica objetoEstado

    if (condicionParaQueOtrosSigan) { // Se verifica con métodos de objetoEstado
        objetoEstado.notifyAll();
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

¿Cuándo utilizar notifyAll() y notify()?

Debe utilizarse notify() cuando se quiere que continúe solo un hilo. Si no, debería utilizarse notifyAll(). Si están bien programados, los hilos reanudados volverán a comprobar si pueden continuar su ejecución y, si no es así, volverán a ejecutar wait() para esperar una nueva ocasión. Por tanto, no debería suponer ningún problema que más de un hilo se reanude con notifyAll(). Pero que esto sea aceptable o no depende del programa.

El uso de notify() en lugar de notifyAll() podría hacer que solo se reanude un hilo que, una vez verificado el estado, decida que no puede continuar y haga wait(), mientras que otro de los hilos podría haber continuado. Esto podría incluso provocar que uno de los hilos quede indefinidamente a la espera para continuar de una notificación que no llega nunca.
Esto no es un interbloqueo o deadlock. De hecho solo hay un objeto de bloqueo implicado. Los hilos no están a la espera de que se libere un objeto de bloqueo, sino de una notificación para seguir que no llega nunca.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

El siguiente programa de ejemplo es similar al de los hilos que incrementan un contador. Pero también existen hilos que decrementan el contador. El contador no puede tener un valor negativo. Si un hilo no puede decrementar el contador, porque su valor es menor que uno, realiza una espera no activa hasta que un hilo incremente el contador y notifique que se puede decrementar el contador.

```
package hilosindec;

class Contador {

    private int cuenta = 0;

    Contador(int valorInicial) {
        this.cuenta = valorInicial;
    }

    synchronized public int getCuenta() {
        return cuenta;
    }

    synchronized public int incrementa() {
        this.cuenta++;
        return cuenta;
    }
    synchronized public int decrementa() {
        this.cuenta--;
        return cuenta;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class HiloIncr implements Runnable {

    private final String id;
    private final Contador cont;

    HiloIncr(String id, Contador c) {
        this.id = id;
        this.cont = c;
    }

    @Override
    public void run() {
        while (true) {

            synchronized (this.cont) {

                this.cont.incrementa();
                this.cont.notify();

                System.out.printf(
                    "Hilo %s incrementa, valor contador: %d\n",
                    this.id,
                    this.cont.getCuenta());
            }
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class HiloDecr implements Runnable {

    private final String id;
    private final Contador cont;

    HiloDecr(String id, Contador c) {
        this.id = id;
        this.cont = c;
    }

    @Override
    public void run() {

        while (true) {

            synchronized (this.cont) {

                while (this.cont.getCuenta() < 1) {
                    System.out.printf(
                        "¡¡ Hilo %s no puede decrementar, valor contador: %d\n",
                        this.id,
                        this.cont.getCuenta());

                    try {
                        this.cont.wait();
                    } catch (InterruptedException ex) {
                    }
                }

                this.cont.decrementa();

                System.out.printf(
                    "Hilo %s decrementa, valor contador: %d\n",
                    this.id,
                    this.cont.getCuenta());
            }
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado


```
class HilosIncDec {

    private static final int NUM_HILOS_INC = 10;
    private static final int NUM_HILOS_DEC = 10;

    public static void main(String[] args) {

        Contador c = new Contador(0);

        Thread[] hilosInc = new Thread[NUM_HILOS_INC];

        for (int i = 0; i < NUM_HILOS_INC; i++) {

            Thread th =
                new Thread(new HiloIncr("INC" + i, c));

            hilosInc[i] = th;
        }
```

```
        Thread[] hilosDec = new Thread[NUM_HILOS_DEC];

        for (int i = 0; i < NUM_HILOS_DEC; i++) {

            Thread th =
                new Thread(new HiloDecr("DEC" + i, c));

            hilosDec[i] = th;
        }

        for (int i = 0; i < NUM_HILOS_INC; i++) {
            hilosInc[i].start();
        }

        for (int i = 0; i < NUM_HILOS_DEC; i++) {
            hilosDec[i].start();
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Actividad propuesta 2.11

Modifica el programa anterior para que exista también un valor máximo para el contador. Verifica que nunca se sobrepasa el valor máximo.

El siguiente programa de ejemplo simula un sistema que controla el paso de personas por un puente, siempre en la misma dirección, para que se cumplan las siguientes restricciones. No pueden pasar más de tres personas a la vez, y no puede haber más de 200 kg de peso en ningún momento. Para realizar la simulación, se modelan las personas como hilos. El tiempo entre la llegada de dos personas es aleatorio entre 1 y 30 s, y para atravesar el puente aleatorio, entre 10 y 50 s. Las personas tienen un peso aleatorio de entre 40 y 120 kg.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

En un objeto de la clase Puente se guarda el estado compartido por todos los hilos: el número de personas que está cruzando el puente y su peso total. Este objeto se pasa a todos los hilos de la clase Persona, en su constructor. Estos lo utilizan como objeto de bloqueo. En esta clase, y esto es importante, se almacena este objeto en una variable de instancia de tipo final. Todos los métodos de esta clase que consultan o modifican su estado son synchronized.

Para simplificar la programación, tiene un método autorizacionPaso que verifica si se cumplen las condiciones para que una persona entre en el puente y, si es así, actualiza su estado. Se invoca este método en un bloque sincronizado sobre el propio objeto.

El método terminaPaso también se invoca en un bloque sincronizado sobre el propio objeto, y después se llama a notifyAll(), para dar la opción de entrar al puente a personas que estén esperando. Con notify se daría opción a una como máximo, lo que sería demasiado restrictivo, porque la salida de una persona podría permitir el paso de más de una.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

La simulación se hace en tiempo real, pero no tiene por qué ser así. Es decir, las esperas en la simulación no tienen por qué ser iguales a las esperas en la realidad. Pero sí hay que guardar la proporcionalidad. Por ejemplo, podrían multiplicarse los tiempos en milisegundos por mil, o multiplicarse por un factor distinto, pero hay que hacerlo siempre por el mismo.

```
import java.util.Random;

class Puente {  // Estado objeto compartido entre los hilos

    private static final int PESO_MAXIMO = 200;
    private static final int MAX_PERSONAS = 3;

    private int peso = 0;
    private int numPersonas = 0;

    synchronized public int getPeso() {
        return peso;
    }

    synchronized public int getNumPersonas() {
        return numPersonas;
    }
```

```
synchronized public int getNumPersonas() {
        return numPersonas;
    }

synchronized public boolean autorizacionPaso(Persona
persona) {

        boolean result;

        if (this.peso + persona.getPeso() <=
Puente.PESO_MAXIMO &&
            this.numPersonas < Puente.MAX_PERSONAS) {

            this.numPersonas++;
            this.peso += persona.getPeso();
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

.

```
            result = true;
        } else {
            result = false;
        }

        return result;
    }

    synchronized public void terminaPaso(Persona persona) {

        this.peso -= persona.getPeso();
        this.numPersonas--;
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
import java.util.Random;

class Persona implements Runnable {

    private final String idPersona;
    private final int peso;
    private final int tMinPaso, tMaxPaso;
    private final Puente puente;

    public int getPeso() {
        return peso;
    }

    Persona(Puente puente,
            int peso,
            int tMinPaso,
            int tMaxPaso,
            String idP) {

        this.puente = puente;
        this.peso = peso;
        this.tMinPaso = tMinPaso;
        this.tMaxPaso = tMaxPaso;
        this.idPersona = idP;
    }
```

```
    @Override
    public void run() {

        System.out.printf(
            "-- %s de %d kg quiere cruzar, en puente %d kg, %d
personas.\n",
            this.idPersona, this.peso,
            puente.getPeso(), puente.getNumPersonas(),
            puente.getNumPersonas() != 1 ? "s" : "");

        // Espera para conseguir autorización

        boolean autorizado = false;

        while (!autorizado) {

            synchronized (this.puente) {

                autorizado = this.puente.autorizacionPaso(this);

                if (!autorizado) {
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado


```
            try {
.
                System.out.printf(
                    "# %s debe esperar.\n",
                    this.idPersona);

                this.puente.wait();

            } catch (InterruptedException ex) {

                System.out.printf(
                    "Interrupción mientras %s espera.\n",
                    this.idPersona);
            }
        }
    }
}

    System.out.printf(
        "-> %s con peso %d puede cruzar, peso en puente %d con %d
personas.\n",
        this.idPersona,
        this.getPeso(),
        this.puente.getPeso(),
        puente.getNumPersonas(),
        this.puente.getNumPersonas() != 1 ? "s" : "");
```

```
Random r = new Random();

// Pasa el puente, y tarda un tiempo en cruzar

int tiempoPaso =
    this.tMinPaso + r.nextInt(this.tMaxPaso - this.tMinPaso + 1);

try {

    System.out.printf(
        "%s va a tardar tiempo %d en cruzar.\n",
        this.idPersona,
        tiempoPaso);

    Thread.sleep(1000 * tiempoPaso);

} catch (InterruptedException ex) {

    System.out.printf(
        "Interrupción mientras %s pasa.\n",
        this.idPersona);
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

.

```
        // Sale del puente

        synchronized (this.puente) {

            this.puente.terminaPaso(this);

            System.out.printf(
                "<- %s sale del puente, puente soporta peso %d, %d
personas.\n",
                this.idPersona,
                this.puente.getPeso(),
                this.puente.getNumPersonas(),
                this.puente.getNumPersonas() != 1 ? "s" : "");

            puente.notifyAll();
        }
    }
    }
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
import java.util.Random;

public class PasoPorPuente {

    public static void main(String[] args) {

        final Puente puente = new Puente();

        int tMinParaLlegadaPersona = 1;
        int tMaxParaLlegadaPersona = 30;

        int tMinPaso = 10;
        int tMaxPaso = 50;

        int minPesoPersona = 40;
        int maxPesoPersona = 120;

        System.out.println(">>>>>>>>>>>> Comienza simulación.");

        Random r = new Random();

        int idPersona = 1;

        while (true) {

            int tParaLlegadaPersona =
                    tMinParaLlegadaPersona +
                    r.nextInt(
                            tMaxParaLlegadaPersona
                            - tMinParaLlegadaPersona + 1);

            int pesoPersona =
                    minPesoPersona +
                    r.nextInt(
                            maxPesoPersona
                            - minPesoPersona + 1);

            System.out.printf(
                    "Siguiente persona llega en %d segundos.\n",
                    tParaLlegadaPersona);

            try {

                Thread.sleep(1000 * tParaLlegadaPersona);

            } catch (InterruptedException ex) {

                System.out.printf(
                        "Interrumpido proceso principal");
            }
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

.

```
                Thread hiloPersona =
                        new Thread(
                                new Persona(
                                        puente,
                                        pesoPersona,
                                        tMinPaso,
                                        tMaxPaso,
                                        "P" + idPersona));

                hiloPersona.start();

                idPersona++;
            }
        }
    }
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Actividad propuesta 2.12

En el programa anterior, el puente solo se puede cruzar en un sentido. A partir de él, crea un programa que controle el paso del puente en ambos sentidos. Como máximo, puede haber cuatro personas en el puente y tres en cada sentido. Puedes no multiplicar los tiempos de espera por 1000 para que la simulación avance más rápidamente.

El siguiente programa de ejemplo es del tipo clásico de productores y consumidores, del que existen muchas variantes. En este tipo de problemas, uno o más procesos productores ponen datos a disposición de uno o más procesos consumidores.

En la formulación de los problemas clásicos de concurrencia se suele utilizar el término genérico proceso. Estos procesos se pueden implementar en cada plataforma con los mecanismos disponibles. En el caso de la plataforma Java, se suele hacer con hilos (threads).

.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Si un proceso consumidor intenta obtener un dato y no hay ninguno disponible, debe esperar a que algún proceso productor introduzca alguno. Los datos se suelen poner en una cola, de la que se extraen los elementos en el mismo orden en que se introducen, según un planteamiento FIFO o first in, first out. Se puede limitar el número de elementos que se admiten en la cola, de manera que, cuando contiene el máximo número de datos, antes de introducir uno nuevo, los productores deben esperar a que algún proceso consumidor retire uno.

En este programa existe un único productor y un único consumidor, y el contenedor puede almacenar un único dato. Por tanto, se alternan las operaciones de introducción de un dato por parte del productor y de extracción por parte del consumidor. La clase Contenedor es genérica, y en este programa se utiliza para almacenar números enteros, en objetos Integer. Se crea un Contenedor<Integer> y se pasa a los hilos productor y consumidor. Se puede ver que los hilos productor y consumidor son prácticamente iguales. Utilizan el propio contenedor como objeto de bloqueo. Cambia la operación que realizan sobre el contenedor (put para introducir un valor y get para obtenerlo) y la condición previa que debe cumplirse para realizarla (que el contenedor no contenga ningún objeto o que contenga alguno)..

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class HiloProductor implements Runnable {

    final Contenedor<Integer> cont;
    String miNombre;

    HiloProductor(
            Contenedor<Integer> cont,
            String miNombre) {

        this.cont = cont;
        this.miNombre = miNombre;
    }

    @Override
    public void run() {

        for (int i = 1;; i++) {

            synchronized (this.cont) {

                while (this.cont.datoDisponible()) {

                    try {

                        this.cont.wait();

                    } catch (InterruptedException ex) {
                    }
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class HiloProductor implements Runnable {

    final Contenedor<Integer> cont;
    String miNombre;

    HiloProductor(
            Contenedor<Integer> cont,
            String miNombre) {

        this.cont = cont;
        this.miNombre = miNombre;
    }

    @Override
    public void run() {

        for (int i = 1;; i++) {

            synchronized (this.cont) {

                while (this.cont.datoDisponible()) {

                    try {

                        this.cont.wait();

                    } catch (InterruptedException ex) {
                    }

                System.out.printf(
                    "Hilo %s produce valor %s.\n",
                    this.miNombre,
                    i);

                    this.cont.put(i);

                    this.cont.notify();
                }
            }
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class Contenedor<T> {

    private T dato;

    synchronized public T get() {

        T result = this.dato;
        this.dato = null;

        return result;
    }

    synchronized public void put(T valor) {

        this.dato = valor;
    }

    synchronized boolean datoDisponible() {

        return (this.dato != null);
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado



Centro Integrado de Formación Profesional nº 1

```
class HiloConsumidor implements Runnable {

    final Contenedor<Integer> cont;
    String miNombre;

    HiloConsumidor(
            Contenedor<Integer> cont,
            String miNombre) {

        this.cont = cont;
        this.miNombre = miNombre;
    }

    @Override
    public void run() {

        while (true) {

            synchronized (this.cont) {

                while (!this.cont.datoDisponible()) {

                    try {
                        this.cont.wait();

                    } catch (InterruptedException ex) {
                    }
                }

                Integer dato = this.cont.get();

                this.cont.notify();

                System.out.printf(
                        "Hilo %s consume valor %d.\n",
                        this.miNombre,
                        dato);
            }
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

```
public class ProductorConsumidor {

    public static void main(String[] args) {

        Contenedor<Integer> almacen =
                new Contenedor<Integer>();

        Thread hprod =
                new Thread(
                        new HiloProductor(
                                almacen,
                                "P"));

        Thread hcons =
                new Thread(
                        new HiloConsumidor(
                                almacen,
                                "C"));

        hprod.start();
        hcons.start();
    }
}
```

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Sin embargo, no es así. El programa se bloquea. No se trata de un interbloqueo, y así se puede verificar con el depurador (se puede ver cómo verificar esto en la sección posterior dedicada a la depuración de programas multihilo). El problema es que, cuando se hace this.cont.notify(); en los hilos Productor y Consumidor, solo se notifica a un hilo, y este podría ser uno que no tiene ninguna posibilidad de continuar con su ejecución. A saber: un hilo consumidor si se acaba de obtener un dato del contenedor o un hilo productor si se acaba de introducir un dato en el contenedor. Este problema se puede solucionar de dos maneras.

La más sencilla es utilizar notifyAll() en lugar de notify() De esta forma, se notifica a todos los hilos suspendidos que pueden continuar su ejecución, y no solo a uno cualquiera. Como acceden al contenedor con un bloque synchronized, lo harán uno a uno. Los que no puedan realizar la operación quedarán bloqueados, pero al menos uno podrá realizarla.

Otra manera es utilizar un objeto de bloqueo para introducir datos y otro para extraerlos.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Existen diversas propiedades que los programas concurrentes (es decir, los que tienen más de un hilo de ejecución) pueden o deben cumplir. Algunas son siempre absolutamente exigibles. Otras aconsejables o exigibles según el grado de confianza o de fiabilidad que se requiera del programa. No es lo mismo un juego para entretenimiento personal que un programa para el control de un electrodoméstico, o incluso para el control de un proceso industrial, cuyo fallo podría tener graves consecuencias económicas o para la salud de las personas.

Seguridad (Safety) La seguridad es una propiedad de los programas concurrentes que consiste en que no producen resultados incorrectos. Se consigue garantizando la exclusión mutua para la ejecución de secciones críticas. En general, es una propiedad siempre exigible.
Vivacidad (Liveness) La vivacidad es una propiedad de los programas concurrentes que consiste en que todos los procesos (o hilos) pueden progresar. Esto significa que, como mínimo, ningún proceso (o hilo) queda indefinidamente a la espera de ejecutar una sección crítica (o, en otras palabras, y de acuerdo con el mecanismo utilizado en este capítulo para implementar la exclusión mutua, de conseguir el bloqueo sobre un objeto de bloqueo). Puede tener un significado específico más restrictivo y, según cuál sea este, los programas concurrentes pueden tener distintos grados o niveles de vivacidad. Estos son, de menor a mayor:

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

1. **Ausencia de interbloqueo (deadlock)**

Ya se ha visto el interbloqueo y cómo evitarlo. En general, es una propiedad siempre exigible.

2. **Ausencia de livelock (interbloqueo activo o estancamiento)**

Este se da cuando no existe interbloqueo (deadlock), pero dos hilos repiten las mismas interacciones a la espera de poder ejecutar una sección crítica, pero sin poder nunca, ninguno de ellos, ejecutarla. Es una situación análoga a la que puede suceder en el mundo real cuando dos personas se encuentran en un pasillo estrecho y, cada vez que una se mueve hacia un lado, la otra hace lo mismo, con lo que siempre se encuentran de frente. Son muy comunes los programas en los que esta se puede dar en teoría, pero es prácticamente imposible que se de en la práctica

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

3. Ausencia de inanición (starvation) o espera finita

La inanición se da cuando un proceso en particular está bloqueado a la espera de que se dé la ocasión de ejecutar una sección crítica, pero esta nunca se da. Un ejemplo sería el control del paso por un puente por el que solo puede pasar un vehículo a la vez. Si está ocupado, quien llega debe esperar en una cola. Cada vehículo tiene una prioridad. Si hay varios esperando, tienen preferencia los de mayor prioridad. Un vehículo puede esperar indefinidamente si siempre llegan vehículos de prioridad más alta. Si no hay prioridades y se elige un vehículo al azar, es teóricamente posible (aunque altamente improbable) que algún vehículo espere indefinidamente.

4. Espera acotada (bounded wait)

Para cualquier proceso (o hilo) a la espera de ejecutar una sección crítica, existe un límite para el número de veces que puede cualquier otro proceso (o hilo) que no está a la espera ejecutar una sección crítica, antes de que pueda hacerlo el proceso a la espera. Si este límite es cero, entonces los procesos pueden ejecutar sus secciones críticas en el mismo orden en que lo solicitan (estrategia FCFS o first come-first served). Esto parece lo más justo, pero no es siempre lo que se quiere. Puede que interese que algunos procesos (o hilos) tengan mayor prioridad que otros. Pero quiere también evitarse la inanición de cualquiera (espera finita), y un tiempo de espera demasiado largo (espera acotada).

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Los programas desarrollados en este capítulo pueden no cumplir algunas de estas propiedades. Pero en general cumplen un mínimo exigible, que es:

- Seguridad. Es decir, ejecución de secciones críticas con exclusión mutua.
- Ausencia de interbloqueo o deadlock.
- Ausencia de interbloqueo activo o livelock.

La ausencia de inanición (starvation) y la espera acotada dependen de la implementación en la JVM de los mecanismos utilizados en los programas. Cuando un hilo ejecuta notifyAll() sobre un objeto de bloqueo, se puede dar la inanición o no dependiendo de la forma en que, a lo largo del tiempo, se seleccione un hilo, para su reanudación, entre todos los bloqueados. Si se selecciona siempre el que lleva más tiempo en espera (espera acotada con límite cero), no se dará la inanición.

Programación de Hilos

2.4. Sincronización de hilos

2.4.4 Compartición de recursos con bloqueo dependiente de su estado

Actividad propuesta 2.13

Prueba el programa anterior, con el cambio sugerido (utilizar notifyAll en lugar de notify). Debes probarlo en situaciones en las que los hilos productores esperan a los hilos consumidores, y en situaciones en las que los hilos consumidores esperan a los hilos productores. Para ello, puedes jugar con el número de hilos de cada tipo, o con tiempos de espera entre sucesivas ejecuciones de las operaciones get y put. Cambiando estos parámetros, en la ejecución del programa se tenderá hacia una situación o la otra. Incluye en comentarios explicaciones de qué cambios hay que hacer para que se produzca cada una de estas situaciones. Es decir, qué valores se pueden asignar a cada uno de estos parámetros para que se produzcan ambas situaciones.

Programación de Hilos

2.4. Sincronización de hilos

2.4.5. Clases thread-safe o seguras para su uso en aplicaciones multihilo

En el programa anterior, las operaciones de sincronización complican los hilos. Un planteamiento alternativo es realizarlas en los métodos de la clase Contenedor. La complejidad de la sincronización entre hilos queda entonces encapsulada en la propia clase, y los hilos pueden llamar a sus métodos sin más. Se dice entonces que la clase es **thread-safe**, es decir, segura para su uso con hilos. En la definición de una clase thread-safe debe, en general, utilizarse la opción throws para propagar las excepciones. El programa queda entonces de la siguiente forma. Para mayor variabilidad en los tiempos, se añaden pausas de duración aleatoria de entre 10 y 50 ms entre sucesivas operaciones. Dado que, en ocasiones, la información que se muestra con System.out.printf puede no aparecer en el orden en que se genera, se muestra un número de secuencia obtenido de una clase Secuencia. Como objeto de bloqueo se utiliza el propio objeto (this).

Programación de Hilos

2.4. Sincronización de hilos

2.4.5. Clases thread-safe o seguras para su uso en aplicaciones multihilo



Centro Integrado de Formación Profesional nº 1

```
import java.util.Random;

class Contenedor<T> {

    private T dato;

    public T get() throws InterruptedException {

        T result;

        synchronized (this) {

            while (!this.datoDisponible()) {
                this.wait();
            }

            result = this.dato;
            this.dato = null;
            this.notifyAll();
        }

        return result;
    }

    public void put(T valor) throws InterruptedException {

        synchronized (this) {

            while (this.datoDisponible()) {
                this.wait();
            }

            this.dato = valor;
            this.notifyAll();
        }
    }

    synchronized boolean datoDisponible() {
        return (this.dato != null);
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.5. Clases thread-safe o seguras para su uso en aplicaciones multihilo


```
import java.util.Random;

class HiloProductor implements Runnable {

    final Contenedor<Integer> cont;
    String miNombre;

    HiloProductor(Contenedor<Integer> cont, String miNombre) {

        this.cont = cont;
        this.miNombre = miNombre;
    }

    @Override
    public void run() {

        Random r = new Random();

        for (int i = 1;; i++) {

            try {

                Thread.sleep(10 + r.nextInt(50 - 10 + 1));

                this.cont.put(i);

                long numSec = Secuencia.getNumSec();

                System.out.printf(
                        "%d-Hilo %s produce valor %s.\n",
                        numSec,
                        this.miNombre,
                        i);

            } catch (InterruptedException ex) {
            }
        }
    }
}
```

Programación de Hilos

2.4. Sincronización de hilos

2.4.5. Clases thread-safe o seguras para su uso en aplicaciones multihilo



Centro Integrado de Formación Profesional nº 1

```
import java.util.Random;

class HiloConsumidor implements Runnable {

    final Contenedor<Integer> cont;
    String miNombre;

    HiloConsumidor(Contenedor<Integer> cont, String miNombre) {

        this.cont = cont;
        this.miNombre = miNombre;
    }

    @Override
    public void run() {

        Random r = new Random();

        while (true) {

            try {

                Thread.sleep(10 + r.nextInt(50 - 10 + 1));

                Integer dato = this.cont.get();

                long numSec = Secuencia.getNumSec();

                System.out.printf(
                        "%d-Hilo %s consume valor %d.\n",
                        numSec,
                        this.miNombre,
                        dato);

            } catch (InterruptedException ex) {
            }
        }
    }
}
```

Programación de Hilos
2.5. Interrupción de hilos

Se puede interrumpir la ejecución de un hilo (Thread) con el método **interrupt**().

Si el hilo se encuentra bloqueado en una llamada a sleep() de la clase Thread o a wait() de la clase Object, se produce una excepción de tipo InterruptedException, que ya ha aparecido en programas de ejemplo anteriores.

Pero todos los hilos deberían permitir su interrupción, aunque no usen nunca ninguno de los métodos anteriores y, por tanto, no se pueda producir en ellos una excepción de tipo InterruptedException. Un hilo puede detectar que ha sido interrumpido (o mejor dicho, que se ha solicitado su interrupción) con el método isInterrupted() de la clase Thread.

El siguiente programa de ejemplo lanza dos hilos. Ambos pueden ser interrumpidos. Uno porque realiza llamadas al método sleep() de la clase Thread y gestiona la InterruptedException correspondiente; y el otro porque, periódicamente, comprueba si ha sido interrumpido con el método isInterrupted() de la clase Thread.

.

Programación de Hilos
2.5. Interrupción de hilos


```
class HiloInterrSleep implements Runnable {

    String nombre;

    public String getNombre() {
        return nombre;
    }

    HiloInterrSleep(String nombre) {
        this.nombre = nombre;
    }

    public void run() {

        try {

            while (true) {
                System.out.println(this.nombre);
                Thread.sleep(1000);
            }

        } catch (InterruptedException ex) {

            System.out.println("***INTERRUMPIDO*** "
                    + this.getNombre());
        }
    }
}
```

```
class HiloInterrNoSleep implements Runnable {

    String nombre;

    public String getNombre() {
        return nombre;
    }

    HiloInterrNoSleep(String nombre) {
        this.nombre = nombre;
    }

    public void run() {

        while (!Thread.currentThread().isInterrupted()) {

            for (long i = 0; i < 1000000000L; i++) {
            }

            System.out.println(this.nombre);
        }

        System.out.println("***INTERRUMPIDO*** "
                + this.getNombre());
    }
}
```

Programación de Hilos
2.5. Interrupción de hilos


```
public class InterrupcionDeHilos {

    public static void main(String[] args) {

        try {

            Thread h1 =
                    new Thread(new HiloInterrSleep("H1"));

            Thread h2 =
                    new Thread(new HiloInterrNoSleep("H2"));

            h1.start();
            h2.start();

            Thread.sleep(5000);

            h1.interrupt();
            h2.interrupt();

        } catch (InterruptedException ex) {

            System.out.println("Hilo principal interrumpido.");
        }
    }
}
```

Programación de Hilos
2.5. Interrupción de hilos

Actividad propuesta 2.14

Se pueden usar los métodos anteriores para hacer que terminen todos los programas desarrollados hasta ahora. Haz que todos los hilos muestren un mensaje cuando capturen la excepción InterruptedException. Si un programa tiene hilos que siguen ejecutándose indefinidamente, haz que todos terminen pasado un tiempo. Utiliza el método interrupt para indicarles que deben terminar, y haz los cambios necesarios para que comprueben cada poco tiempo si han sido interrumpidos, con isInterrupted..

Programación de Hilos
2.6. Prioridades

Cada hilo se crea con una prioridad determinada. Por defecto, se crea con la prioridad del hilo que lo crea. Se puede cambiar con setPriority() y se puede consultar con getPriority()-try().

La prioridad normal es NORM_PRIORITY, la máxima es MAX_PRIORITY, y la mínima es MIN_PRIORITY.

Cuando el planificador a corto plazo tiene que elegir el siguiente hilo que ejecutar, elige uno de los que tienen la máxima prioridad de entre todos los que están en estado listos para ejecución.

El siguiente programa lanza un hilo con cada uno de los posibles valores para la prioridad, de Thread.MIN_PRIORITY a Thread.MAX_PRIORITY. Son todos iguales, aparte de la distinta prioridad que tiene cada uno. Cada uno de ellos incrementa repetidamente su propio contador. Al cabo de unos segundos, se interrumpen todos los hilos y se muestra la cuenta a la que ha llegado cada uno.

Programación de Hilos
2.6. Prioridades



Centro Integrado de Formación Profesional nº 1

```
import java.util.ArrayList;

public class PrioridadDeHilos {

    public static void main(String[] args) {

        ArrayList<Integer> prioridades =
                new ArrayList<Integer>();

        for (int i = Thread.MIN_PRIORITY;
                i <= Thread.MAX_PRIORITY;
                i++) {

            prioridades.add(i);
        }

        try {

            ArrayList<HiloConPrioridad> hilos =
                    new ArrayList<HiloConPrioridad>();

            for (Integer prior : prioridades) {

                // Crea hilos

                HiloConPrioridad hilo =
                        new HiloConPrioridad();

                hilo.setPriority(prior);

                hilos.add(hilo);
            }

            for (Thread hilo : hilos) {

                // Arranca hilos

                hilo.start();
            }

            int duracionSegundos = 10;

            System.out.printf(
                    "Se deja correr los hilos durante %d segundos.\n",
                    duracionSegundos);

            Thread.sleep(1000 * duracionSegundos);

            for (Thread hilo : hilos) {

                // Interrumpe hilos

                hilo.interrupt();
            }

            for (HiloConPrioridad hilo : hilos) {

                // Muestra cuenta para cada hilo

                System.out.printf(
                        "Contador para proceso con prioridad %d: %d\n",
                        hilo.getPriority(),
                        hilo.getContador());
            }

        } catch (InterruptedException ex) {

            System.out.println(
                    "Hilo principal interrumpido.");
        }
    }
}
```

Programación de Hilos
2.6. Prioridades



Centro Integrado de Formación Profesional nº 1

```
import java.util.ArrayList;

class HiloConPrioridad extends Thread {

    long contador = 0;

    public long getContador() {
        return contador;
    }

    private void incContador() {
        contador++;
    }

    public void run() {

        while (!Thread.currentThread().isInterrupted()) {
            this.incContador();
        }
    }
}
```

Programación de Hilos
2.6. Prioridades



Centro Integrado de Formación Profesional nº 1

```
import java.util.ArrayList;

class HiloConPrioridad extends Thread {

    long contador = 0;

    public long getContador() {
        return contador;
    }

    private void incContador() {
        contador++;
    }

    public void run() {

        while (!Thread.currentThread().isInterrupted()) {
            this.incContador();
        }
    }
}
```

Programación de Hilos

2.8. Mecanismos de alto nivel para concurrencia

Java proporciona diversos mecanismos de alto nivel para concurrencia en el paquete **java.util.concurrent.** Vale la pena tenerlos en cuenta, porque pueden simplificar el desarrollo de aplicaciones concurrentes. Con ellos se puede evitar esfuerzo innecesario y mejorar la calidad del software desarrollado, utilizando mecanismos de alto nivel y patrones ya disponibles y probados exhaustivamente. Este paquete es muy amplio, por lo que solo se explicarán algunos componentes sencillos que puedan ser de utilidad para desarrollar programas similares a los programas de ejemplo mostrados en este capítulo.

2.8.1. Colecciones concurrentes

La interfaz **BlockingQueue** define una cola FIFO (first-in-first-out) que bloquea procesos que intentan extraer un elemento cuando la cola está vacía, a la espera de que haya algún elemento disponible en ella. También permite especificar un número máximo de elementos, de manera que se bloquean procesos que intentan añadir elementos cuando la cola crece ese número de elementos. También permite especificar un tiempo máximo para el bloqueo. Esta interfaz tiene varias subclases que la implementan sobre distintas estructuras de datos.

ConcurrentMap es una subinterfaz de java.util.Map con operaciones atómicas para eliminar o reemplazar un par clave-valor solo si la clave está presente, o para añadir un par clave-valor solo si la clave no está presente. Una clase que implementa esta interfaz es ConcurrentHashMap, que es análoga a HashMap.

2.8.2. Variables atómicas

El paquete **java.util.concurrent.atomic** incluye clases que proporcionan operaciones atómicas sobre datos no estructurados, como números enteros o decimales con punto flotante.

Tienen una gran variedad de métodos que permiten recuperar el valor actual y, a la vez, y de forma atómica, realizar diversas operaciones con él, tales como incrementar, decrementar, sumar y restar.

Programación de Hilos

2.8. Mecanismos de alto nivel para concurrencia

Algunas de las clases incluidas son:

AtomicBoolean
AtomicInteger
AtomicIntegerArray
AtomicLong
AtomicLongArray

2.8.3. Números aleatorios

ThreadLocalRandom proporciona números aleatorios para aplicaciones multihilo, en sustitución de Math.random().

Se puede obtener un número aleatorio con el siguiente código:

```
int r = ThreadLocalRandom.current().nextInt(0, 100);
```

El número devuelto estaría entre:

0 y 100 - 1 = 99

Existen métodos similares:

nextBoolean()
nextDouble()
nextFloat()
nextLong()
etc.

Actividades propuestas 2.15
Utiliza alguna clase que implemente la interfaz BlockingQueue para hacer lo mismo que en programas de ejemplo o actividades propuestas para problemas de tipo productor-consumidor planteadas hasta ahora o en los ejercicios del final de este capítulo.

2.16
Utiliza la clase AtomicInteger para hacer lo mismo que el programa inicial, en el que varios hilos incrementaban un mismo contador, hasta hacerlo llegar entre todos a un valor determinado.

Programación de Hilos

RESUEMEN

La ejecución de un proceso comienza con un único hilo, pero este puede crear nuevos hilos sobre la marcha. Los distintos hilos de un mismo proceso comparten memoria y código de programa.

El acceso a los objetos compartidos por distintos hilos de un mismo proceso debe controlarse mediante mecanismos de sincronización, para evitar errores que podrían darse si más de un hilo modifica a la vez el estado de dichos objetos.

Los hilos en Java se representan mediante objetos de la clase Thread. Se pueden crear hilos a partir de objetos de cualquier clase que implemente la interfaz Runnable. La propia clase Thread implementa esta interfaz.

El proceso que realiza un hilo se implementa en el método run. Se inicia la ejecución de un hilo con el método start, que a su vez ejecuta el método run.

El método join suspende la ejecución del hilo actual a la espera de que termine la ejecución del hilo para el que se invoca.

Programación de Hilos

RESUEMEN

La palabra clave synchronized hace que un hilo solo pueda ejecutar un bloque de código tras obtener el bloqueo de un objeto de bloqueo, que se libera una vez ejecutado el bloque. Si el objeto de bloqueo está bloqueado, se suspende la ejecución del hilo a la espera de conseguir el bloqueo del objeto de bloqueo. De esta manera, se consigue que el bloque de código se ejecute en exclusión mutua con otros hilos. Se puede aplicar para métodos no estáticos (en cuyo caso, el objeto de bloqueo es this), para métodos estáticos (en cuyo caso, el objeto de bloqueo es la clase a la que pertenece el método) y para bloques de código cualesquiera (en cuyo caso, se especifica explícitamente el objeto de bloqueo).

El interbloqueo o deadlock se produce cuando dos o más hilos mantienen bloqueado un conjunto de objetos de bloqueo, y su ejecución está suspendida a la espera de conseguir bloquear cada uno un nuevo objeto de bloqueo, que no pueden bloquear porque ya están bloqueados por otros hilos.

Se puede evitar el interbloqueo si se establece un orden para los objetos de bloqueo, y todos los hilos los bloquean según este orden.

Programación de Hilos

RESUEMEN

Si, una vez conseguido el bloqueo sobre un objeto de bloqueo obj en un bloque de código synchronized(obj), un hilo comprueba que no se dan las condiciones necesarias para continuar con su ejecución, puede ejecutar wait() sobre el objeto de bloqueo con obj.wait(). Con ello, su ejecución queda suspendida y, en espera no activa, mientras que otro objeto no ejecute obj.notifyAll() u obj.notify(). notifyAll() reanuda todos los procesos a la espera en el objeto de bloqueo, mientras que notify() reanuda solo uno de ellos.

Programación de Hilos

EJERCICIOS

Notas generales para todos estos ejercicios

Es importante que los objetos de clases con métodos synchronized que se pasan en el constructor de los hilos (es decir, de objetos de clases que implementan la interfaz Runnable, que contienen la información de estado compartida entre todos los hilos, y que se utilizan como objetos de bloqueo, se asignen a variables de instancia de tipo final. Esto garantiza que, una vez asignada la referencia a un objeto, esta no cambia. Si lo hiciera, se invalidarían todos los bloqueos que existan sobre el objeto.
Deben mostrarse mensajes en la pantalla para las operaciones de todos los hilos, y para todos los eventos que suceden durante la ejecución de los hilos, para que se pueda comprender la secuencia de eventos durante la simulación, conocer el estado en cada momento y, por tanto, que se pueda validar el funcionamiento de los programas en las diversas situaciones que se puedan dar.
Cuando se habla de procesos debe entenderse que es de manera genérica, según la terminología habitual de la programación concurrente, y deben implementarse como hilos en Java.

Programación de Hilos

EJERCICIOS

EJERCICIO 1

Varios hilos (por ejemplo, cuatro) realizan una colecta. En un tiempo aleatorio entre 10 y 200 ms, consiguen una cantidad de entre 4 y 25. La colecta termina cuando se llega a una cantidad de 2.000. Utilizar una clase Colecta que almacene y permita consultar la cantidad recogida hasta el momento. El hilo principal crea un objeto de tipo Colecta, crea los hilos y les pasa este objeto, que es compartido entre todos y, por tanto, sus métodos deben protegerse de manera apropiada con métodos synchronized.

Programación de Hilos

EJERCICIOS

EJERCICIO 2

Se tienen 3 depósitos de pintura, cada uno de un color primario, a saber: cian, magenta y amarillo. Se tienen 3 personas que intentan preparar, cada una, pintura de un color secundario distinto. Estos son rojo (mezcla de amarillo y magenta), azul (mezcla de magenta y cian) y verde (mezcla de amarillo y cian). Para preparar un color secundario, una persona debe tener acceso en exclusiva a los dos colores primarios necesarios. Cada persona debe implementarse mediante un hilo que, en un bucle sin fin, haga lo siguiente: obtener acceso en exclusiva al depósito de los dos colores primarios que necesita, mantenerlo durante un tiempo aleatorio (por ejemplo, entre 100 y 500 ms) y esperar un tiempo aleatorio antes de volver a intentar preparar de nuevo el color, que podría estar, por ejemplo, entre 1 y 2 s. Hay que tener especial cuidado en realizar el bloqueo de los depósitos de pintura (como se ha dicho, uno por cada color primario), de manera que no se pueda nunca producir un interbloqueo. Para una prueba más exhaustiva, se puede probar este programa sin pausas para el tiempo de estudio ni para el tiempo de descanso, durante varios segundos.

Programación de Hilos

EJERCICIOS

EJERCICIO 3

Implementa una solución sencilla al problema de la comida de los 5 filósofos de Dijkstra que evite el interbloqueo. Cinco filósofos pasan la vida pensando en una mesa redonda, y solo dejan de pensar de vez en cuando para comer de un cuenco de espaguetis. Para ello, deben tomar los palillos que tienen a ambos lados, primero uno y luego el otro. Por supuesto, no pueden coger un palillo si lo está utilizando otro filósofo. Entonces, deben esperar a que termine y lo suelte. Una vez tomado un palillo, no lo sueltan hasta que no consiguen el otro palillo. Entonces, comen y, cuando han terminado de comer, dejan ambos palillos de nuevo en la mesa.

Se recomienda numerar a los filósofos de 0 a 4. El filósofo i come con los palillos i a su izquierda e (i + 1) mod 5 (a su derecha).

Debe tenerse especial cuidado para que la solución implementada elimine toda posibilidad de interbloqueo. ¿Cómo lo has hecho?
¿Sería posible, con la solución que has implementado, que algún filósofo no consiguiera comer nunca, por improbable que fuera? Si es así, explica el escenario en que esto sucedería.

Programación de Hilos

EJERCICIOS

EJERCICIO 4

Se tienen, como en un ejercicio anterior, varios hilos que realizan una colecta, pero no se puede recoger más dinero una vez que se ha llegado a una cantidad prefijada (por ejemplo, 2.000). Además, hay hilos consumidores que reciben una cantidad de dinero, por ejemplo, entre 10 y 40, a intervalos de tiempo aleatorios entre 20 y 300 ms. Los hilos recolectores y los consumidores se ejecutan indefinidamente. Los hilos recolectores no pueden recoger dinero cuando se ha llegado a la cantidad máxima. Deben esperar a que los hilos consumidores retiren dinero de manera que puedan aportar la cantidad que han recogido. Si los hilos consumidores no pueden retirar la cantidad de dinero que necesitan, porque no hay suficiente recolectado, deben esperar a que los hilos recolectores aporten suficiente dinero. En ambos casos, la espera debe ser no activa. Deben probarse ambas situaciones (los hilos recolectores esperan porque se ha pasado del máximo que se puede recoger, y los hilos consumidores esperan porque no se ha recolectado suficiente dinero), y para ello se puede jugar con el número de hilos consumidores. Incluir en comentarios una breve explicación acerca del número de hilos consumidores con los que se puede probar cada una de estas situaciones.

Programación de Hilos

EJERCICIOS

EJERCICIO 5

Existen 10 procesos que necesitan obtener una cantidad de entre 10 y 25 de un recurso para realizar una actividad. El recurso es compartido y lo proporciona un proceso suministrador. Una vez que un proceso obtiene la cantidad de producto que necesita, tarda un tiempo aleatorio de entre 100 y 500 ms en realizar una actividad con él. Si no obtiene la cantidad que necesita, debe esperar hasta que esté disponible. El proceso suministrador repone cada segundo una cantidad de producto de entre 50 y 120, pero nunca puede haber más de 600, por limitaciones de espacio de almacenamiento. Realiza una simulación de estos procesos utilizando hilos en Java.

Programación de Hilos

EJERCICIOS

EJERCICIO 6

Igual que el programa anterior, pero la simulación debe terminar pasados 15 segundos. Los hilos no deben terminar abruptamente, sino que deben terminar las actividades que tengan en marcha. Es decir, que si un hilo ha obtenido la cantidad de recurso necesaria para realizar la actividad, debe terminar de realizarla y luego terminar su ejecución.

Programación de Hilos

EJERCICIOS

EJERCICIO 7

Existen 4 personas que trabajan un tiempo aleatorio entre dos y cuatro horas, se preparan un bocadillo, y vuelven a trabajar, y así todo el tiempo. Cada bocadillo necesita 2 hojas de lechuga, 1 tomate y 100 g de jamón. Inicialmente, se tienen 6 hojas de lechuga, 3 tomates y 500 g de jamón. Existe un proceso para reponer cada una de estas tres cosas. La lechuga se repone cada 8 horas; los tomates, cada 5 horas; y el jamón, cada 12 horas, y se repone hasta la cantidad inicial. Realiza una simulación.

Programación de Hilos

EJERCICIOS

EJERCICIO 8

Escribe un programa para el problema clásico de los procesos productores y consumidores, pero utilizando un contenedor. Un proceso productor almacena datos en un contenedor, y un proceso consumidor obtiene los datos del mismo contenedor. Los datos deben consumirse en el mismo orden en que se producen, según el modelo de cola FIFO (first in, first out). Los datos producidos deben ser números secuenciales, empezando por 1 e incrementando de uno en uno. El contenedor debe ser un ArrayList.

Debe protegerse el acceso al contenedor, dado que la clase ArrayList no es thread-safe. Debe crearse una clase ColaSincronizada con métodos get y put. Todos los mecanismos de sincronización deben estar implementados en los métodos de la clase, por lo que los productores y consumidores, sencillamente, llaman a los métodos get y put.

Programación de Hilos

EJERCICIOS

EJERCICIO 9

Como el ejercicio anterior, pero con varios productores y varios consumidores. Cada productor debe producir una secuencia de datos consistente en un identificador constante, seguido de un número secuencial. Por ejemplo: el primer productor podría producir la secuencia P1_1, P1_2, ..., el segundo P2_1, P2_2.... Probar con distintos valores para el número de productores y de consumidores.

EJERCICIO 10

Como el ejercicio anterior, pero el contenedor no puede almacenar más de 10 datos. Si contiene 10 datos, los procesos productores deben esperar hasta que se consuma un dato antes de añadir uno nuevo.

Programación de Hilos

EJERCICIOS

EJERCICIO 11

Se tiene un aparcamiento con 50 plazas, numeradas de 1 a 50, al que intentan acceder coches continuamente. La entrada de coches se puede simular con el programa principal que, cada cierto tiempo (aleatorio, por ejemplo entre 500 y 3000 ms), crea un hilo que simula un coche. Cada coche se crea con una matrícula distinta. Cada coche intenta conseguir acceso al aparcamiento, indicando su matrícula para ello, que queda registrada para la plaza de aparcamiento libre, si la hay, que se le asigna. Pasado un tiempo aleatorio (por ejemplo, de 10 a 20 s), el coche abandona el aparcamiento. Desarrolla un programa que simule el aparcamiento.

EJERCICIO 12

Lo mismo que el anterior problema, pero en el aparcamiento pueden entrar camiones, y un camión debe aparcar en dos plazas contiguas, una impar y otra par (siendo la impar la menor de ellas, dado que entre una plaza par y la siguiente impar hay una columna). Para simular la entrada de coches y camiones, el programa principal puede obtener un número aleatorio al azar, y crear un hilo que represente un coche o un camión, dependiendo del valor obtenido. En cualquier caso, los camiones deberían entrar con mucha menos frecuencia que los coches.